

COS 135 Individual Assignment 8

Due: Saturday 04/25/26 End of the day

(100pts) write a C program for the following task and submit your source code (you must submit .c file without compilation errors). **DO NOT MODIFY THE GIVEN DATASET (i.e., cars.csv).**

Note: study the given code samples that shows how to read data from the dataset / file, create struct arrays, and sort a second struct pointer array based on the values in the original dataset.

What to submit:

- Please submit a .c source file.

[-10 pts each] Source codes should be able to compile and execute without errors or warnings:

- **[-10 pts if any compiler errors]** Source codes should be able to compile without any errors
- Always compile your source codes with -Wall (enable all warnings) switch to find all the compiler warnings and fix them (see example below).

```
$ gcc mycode.c -o myprogram -Wall
```

[-10 pts each] **Comments** are required in the following locations (in each C source code):

- **[-2 pts]** At the top of the source code, comment your name and insert a short program description
- **[-4 pts]** Comment the purpose of each variable.
- **[-4 pts]** Comment major sections of your code such as inputs, functions, and outputs.

Program Design: Your program is a professional document and must be neat and easy to read.

All programs should follow these specifications.

- Comments should be aligned and entered in a consistent fashion
- Blank lines should be added to aid readability
- Code within blocks should be indented
- Comments should not contain spelling mistakes
- Variable names should be meaningful
- Define functions where necessary
- Take time to design your code (program flow and functions) before start coding

(100pts) File **cars.csv** contains a dataset on various popular car models in the US market from 1970 to 1982. The dataset is saved in a .csv (Comma Separated Values) file. You may use the Linux *cat* command to print its content on the terminal to see comma-separated values (.csv) and its format. Alternatively, you can also use any plain text editor (VIM, Nano) or spreadsheet application such as MS Excel to observe the content (*note: spreadsheet applications usually do not show the comma-separated format; only shows the cells*). More information on .csv format: https://en.wikipedia.org/wiki/Comma-separated_values

Your task is to develop a C program to analyze the dataset. Follow the steps below.

1. First, observe the dataset (especially the columns and their headings). The first two rows are used for column headers and datatypes.

To see only the first 10 rows, you can use the **head** command in the Linux terminal as below:

```
$ head -10 cars.csv
```

2. Define a C struct to save an instance of the dataset (i.e., one row). You may use the given data types in the second row of the dataset when defining the struct members.

e.g.,

```
struct car {  
    // struct member 1  
    // struct member 1  
    ...  
    // struct member n  
};
```

3. Define a dynamically allocated struct array to store all the information from cars.csv (your program should read all the information row by row and fill the corresponding struct array). Each row is an element in the structs array. The data structure (struct array) should grow when the program reads info from the cars.csv. The following example defines a single element struct in the memory.

```
struct car *cars = (struct car*) malloc(sizeof(struct car));
```

4. Once the dataset is read into the structs array, define an array of struct pointers (N = size = total number of records in the dataset). This new array of struct pointers should hold the records in ascending order (sort) based on MPG values. Note: order of the original dataset should not be changed.

e.g.,

```
struct car *car_ptrs[N];
```

5. Finally, output the most efficient and least efficient cars based on the MPGs (i.e., cars pointing from first and last pointers in the **car_ptrs** struct pointer array).

Note: please refer to the given code samples that show how to read data from the dataset, create struct arrays, and sort a second struct pointer array based on the values in the original dataset.

Define functions where necessary. Show only up to 2 decimal points.

Program output:

```
Least efficient car: Ford F250,  
    MPG: 10.00, Cylinders: 8, Horsepower: 215.00, Weight: 4615.00, Model: 70, Origin: US  
Most efficient car: Mazda GLC,  
    MPG: 46.60, Cylinders: 4, Horsepower: 65.00, Weight: 2110.00, Model: 80, Origin: Japan
```